DECP

Department Engagement & Career Platform

Mini Project Report

Project Team

Index No.	Name	Role
E/20/089	Edirimanna Y.H.	Enterprise Architect / Security & DevOps Architect
E/20/361	Senadheera Y.H.	Solution Architect
E/20/366	Senevirathna A.P.B.P	Application Architect

Live Demo: https://co-528-labs-oito.vercel.app/
GitHub: https://github.com/YasiruHarinda/CO528-Labs/tree/main/mini_project

1. Introduction

1.1 Project Overview

Department Engagement & Career Platform (DECP) is a full-stack web and mobile application developed for the Department of Computer Engineering, University of Peradeniya. The application allows current students, alumni, and staff of the department to connect, collaborate, and engage with the community..

1.2 Objectives

- Design and implement a Service-Oriented Architecture (SOA) with clearly defined API boundaries
- Build functional web and mobile clients that consume the same backend APIs
- Deploy backend services and database to cloud infrastructure
- Implement core features: user management, feed, jobs, events, research, messaging, and analytics
- Demonstrate quality attribute considerations including security, scalability, and maintainability

1.3 Scope

The platform targets three primary user groups:

- Current Students - share posts, apply for jobs, RSVP to events, collaborate on research
- Alumni - post job/internship opportunities, share professional updates, mentor students
- Department Staff - manage events, post announcements, view analytics

2. Architecture Design

2.1 Service-Oriented Architecture (SOA)

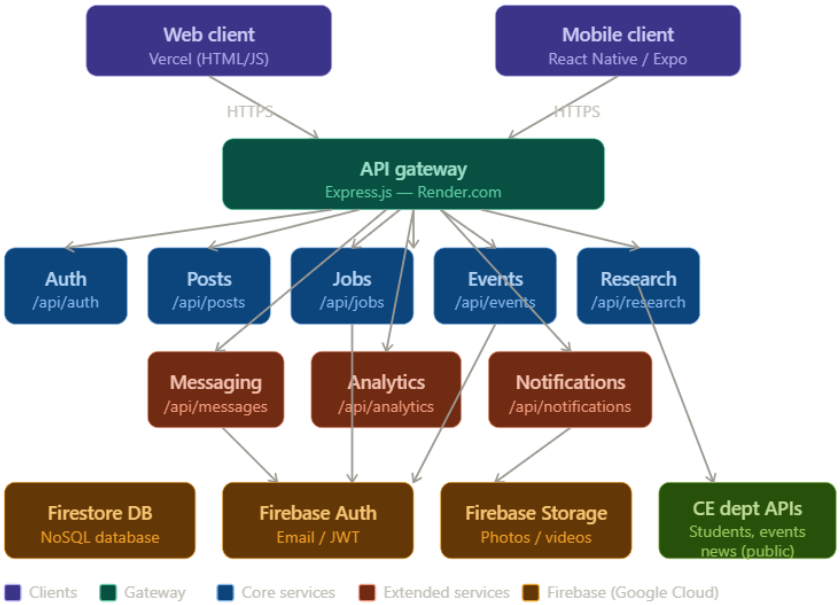
DECP follows a Service-Oriented Architecture where each functional domain is implemented as an independent service with well-defined REST API endpoints. All services communicate through a central API Gateway and are consumed by both web and mobile clients.

2.1.1 Services Overview

Service	Base Endpoint	Responsibility
Auth Service	/api/auth	User registration, login, JWT token management
Posts Service	/api/posts	Feed posts, likes, comments, media upload
Jobs Service	/api/jobs	Job/internship listings, applications
Events Service	/api/events	Department events, RSVP management
Research Service	/api/research	Research projects, collaboration invites
Messaging Service	/api/messages	Direct messaging between users
Analytics Service	/api/analytics	Usage stats, active users, job metrics
Notifications Service	/api/notifications	Push notifications via Web Push (VAPID)

2.1.2 API Design Principles

- RESTful conventions with consistent JSON responses
- JWT-based stateless authentication on all protected endpoints
- HTTP status codes used correctly (200, 201, 400, 401, 403, 404, 500)
- Input validation on all POST/PUT endpoints
- Error responses include descriptive messages for debugging



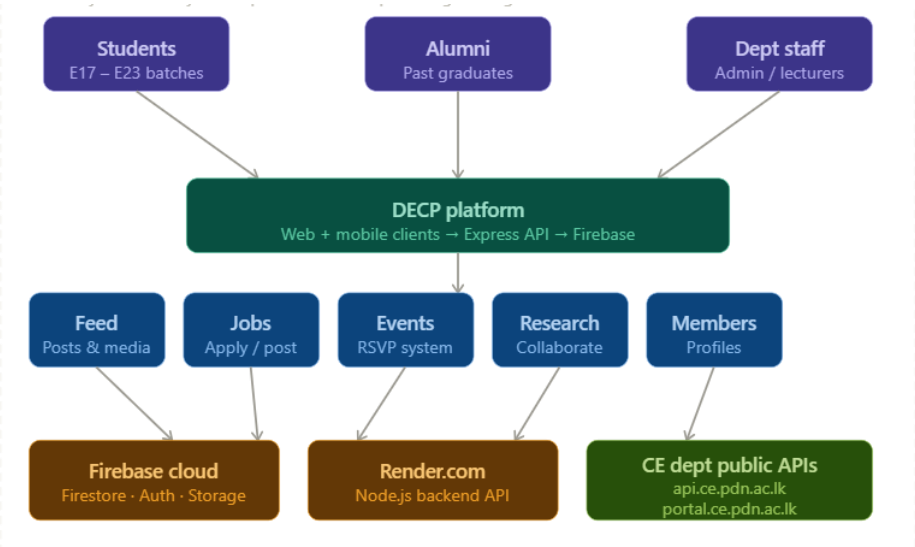
2.2 Enterprise Architecture

The enterprise architecture shows how DECP integrates with existing departmental systems and user roles at the University of Peradeniya.

2.2.1 Integration with CE Department APIs

- Student data fetched from: api.ce.pdn.ac.lk/people/v1/students/
- Real department events from: portal.ce.pdn.ac.lk/api/events/v2/
- Live news feed from: portal.ce.pdn.ac.lk/api/news/v1/

These integrations ensure DECP displays real, live data from the CE department without duplicating data storage.



2.2.2 User Roles and Permissions

Role	Permissions
Student	View feed, post updates, apply for jobs, RSVP events, join research
Alumni	All student permissions + post jobs/internships, mentor students
Admin	All permissions + manage users, post events, view analytics dashboard

2.3 Product Modularity Architecture

2.3.1 Core Modules (Required)

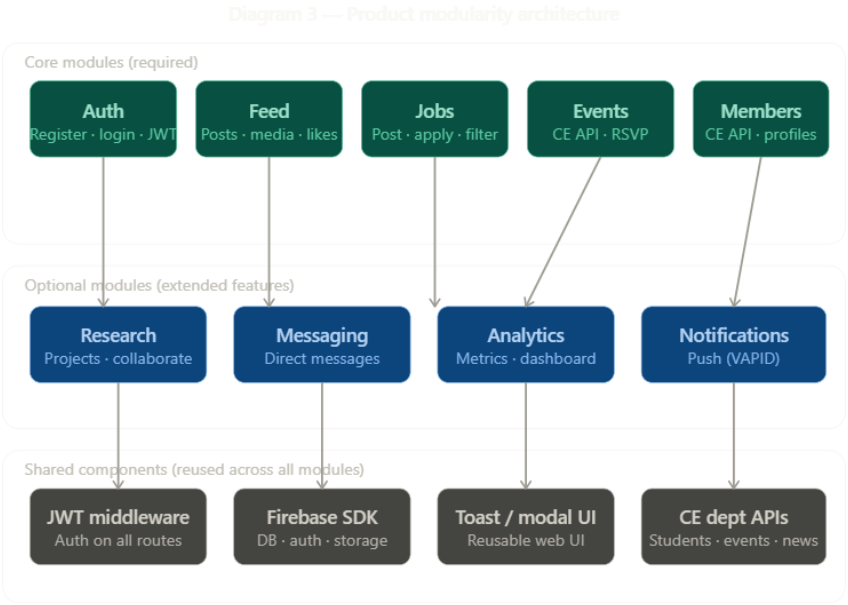
- User Authentication - Registration, Login, JWT Session Handling
- Feed & Media Posts - Text Post, Photo/Video Upload, Like, Comment, Share
- Jobs & Internships - Post Job/Internship, One-Click Apply Job
- Events & Announcements - CE Department Events and RSVP System
- Members Directory - Student Profile from CE Dept API and Photos

2.3.2 Optional/Extended Modules

- Research Collaboration - Project Creation and User Invitation
- Messaging - Direct Message between Two Platform Users
- Analytics Dashboard - Active User Count, Post Engagement, Job Application Count
- Push Notifications - Browser Push Notifications via VAPID

2.3.3 Shared Components

- Authentication Middleware - Shared by all services for JWT verification
- Firebase Admin SDK - Shared by all services for Firebase integration
- Toast Notification System - Shared by all services for UI component reuse
- Modal System - Shared by all services for UI component reuse in jobs and events and research



2.4 Deployment Architecture

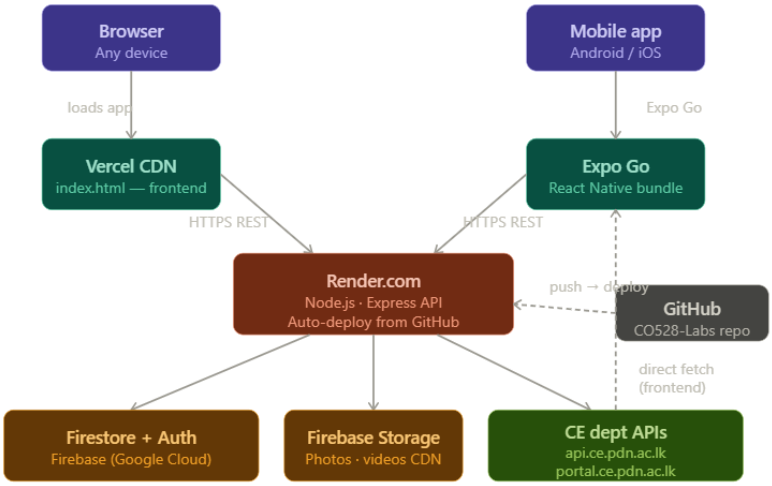
2.4.1 Cloud Infrastructure

Component	Platform	Details
Web Frontend	Vercel	Single HTML file, CDN-distributed globally
Backend API	Render.com	Node.js Express app, auto-deploy from GitHub
Database	Firebase Firestore	NoSQL cloud database, real-time capable
Authentication	Firebase Auth	Email/password auth with JWT tokens
Media Storage	Firebase Storage	Photo/video uploads, public CDN URLs
Mobile App	Expo / React Native	WebView wrapper, runs on Android & iOS
Code Repository	GitHub	github.com/YasiruHarinda/CO528-Labs

2.4.2 Deployment Flow

- Developer pushes code to GitHub main branch
- Vercel auto-detects change → rebuilds and deploys frontend in ~1 minute
- Render auto-detects change → rebuilds Node.js backend in ~3 minutes
- Firebase Firestore and Storage always available — no deployment needed
- Mobile app updated via Expo OTA (Over The Air) updates

Diagram 4 — Cloud deployment architecture



Region: asia-south1 (Mumbai) — closest to Sri Lanka

CI/CD: GitHub push → Vercel + Render auto-deploy within 3 minutes

3. Implementation Details

3.1 Technology Stack

Layer	Technology	Justification
Backend	Node.js + Express	Lightweight, fast, large ecosystem, ideal for REST APIs
Database	Firebase Firestore	NoSQL, serverless, real-time, free tier sufficient for project
Authentication	Firebase Auth + JWT	Industry standard, secure, stateless token-based auth
Web Frontend	HTML/CSS/JavaScript	Single-file SPA, no build step, easy deployment
Mobile	React Native + Expo	Cross-platform, same codebase for Android and iOS
Push Notifications	Web Push + VAPID	Standard browser push, no third-party service required
Media Storage	Firebase Storage	5GB free tier, integrated with Firebase Auth

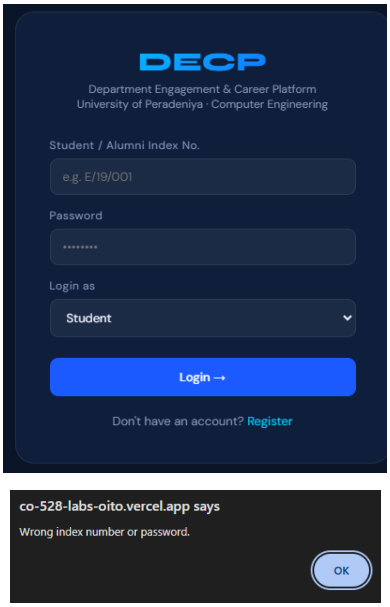
3.2 Backend Implementation

3.2.1 Project Structure

```
decp-backend/
├── index.js              — Express app entry point
├── firebase.js           — Firebase Admin SDK initialization
├── middleware/auth.js    — JWT verification middleware
├── utils/notify.js       — Push notification helper
└── routes/
    ├── auth.js           — Register, login, user profile
    ├── posts.js          — Feed CRUD, likes, comments, media
    ├── jobs.js           — Job listings, applications
    ├── events.js         — Events, RSVP
    ├── research.js       — Research projects
    ├── messages.js       — Direct messaging
    ├── analytics.js      — Platform metrics
    └── notifications.js  — Push notification subscriptions
```

3.2.2 Authentication Flow

- User registers with index number (e.g. E/20/089) and password
- Index number converted to university email format: e20089@eng.pdn.ac.lk
- Firebase Auth creates user account with email/password
- User profile saved to Firestore users collection with name, batch, role
- JWT token generated with 7-day expiry and returned to client
- All subsequent API requests include JWT in Authorization header
- Backend middleware verifies JWT before processing any protected route



3.2.3 Media Upload Flow

- User selects photo/video in web client
- Frontend sends file to POST /api/posts/upload using FormData
- Backend receives file via multer (memory storage)
- File uploaded to Firebase Storage under posts/ folder
- Firebase returns public CDN URL
- URL saved in Firestore post document as mediaUrl field
- On reload, posts fetched from Firestore include mediaUrl images persist

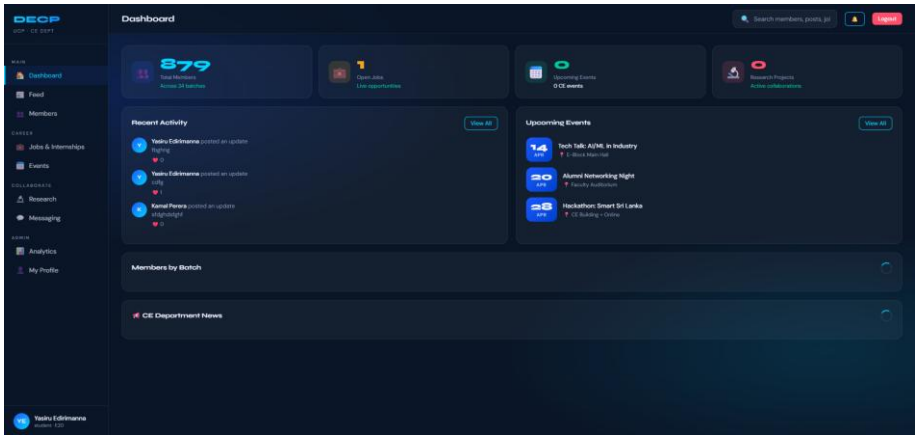
3.3 Web Client Implementation

3.3.1 Architecture

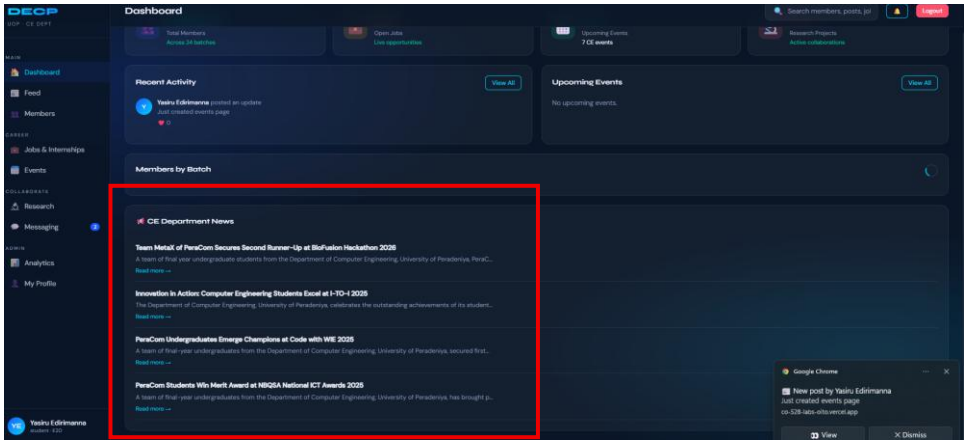
The web client is implemented as a Single Page Application (SPA) in a single HTML file with embedded CSS and JavaScript. This approach eliminates build complexity while maintaining full SPA behavior through client-side routing.

3.3.2 Key Features Implemented

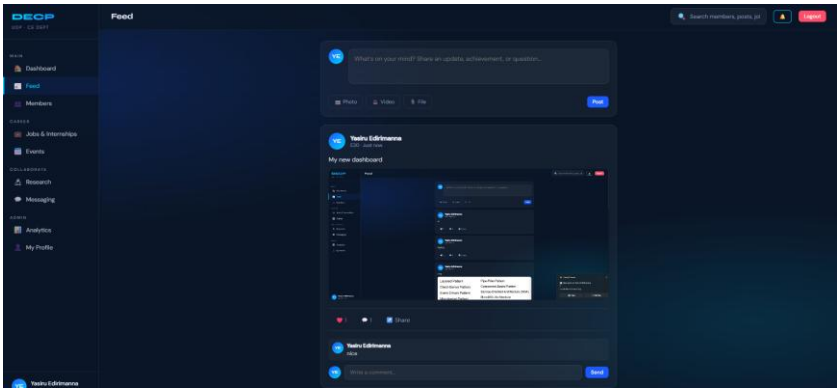
- Dashboard - Real-time stats from CE API and backend, animated counters, live news

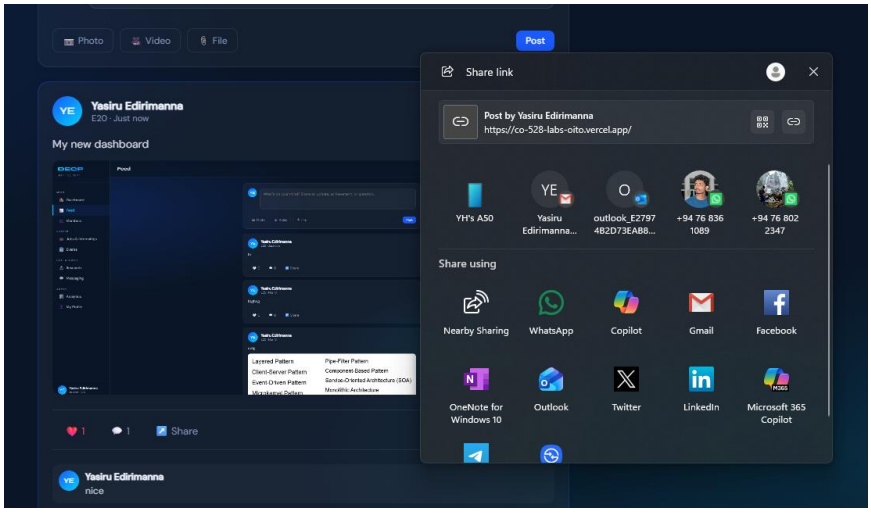


Live news

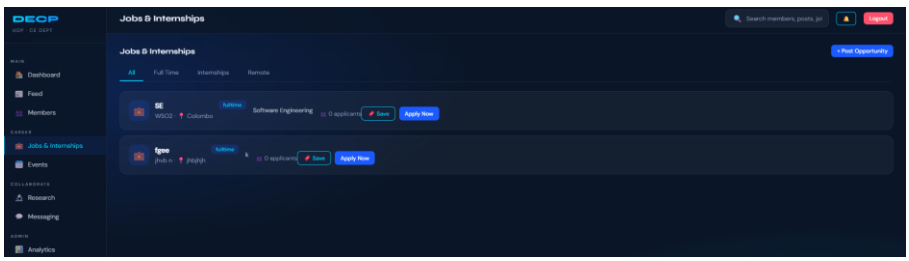


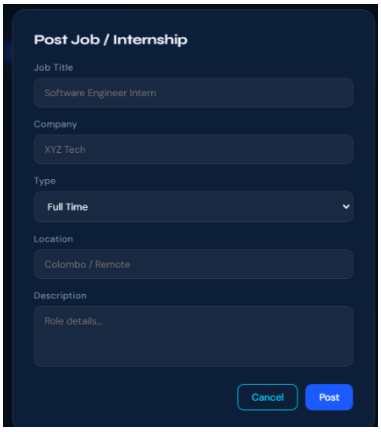
- Feed - Create posts with photo/video, like, comment, share using data-attribute event delegation



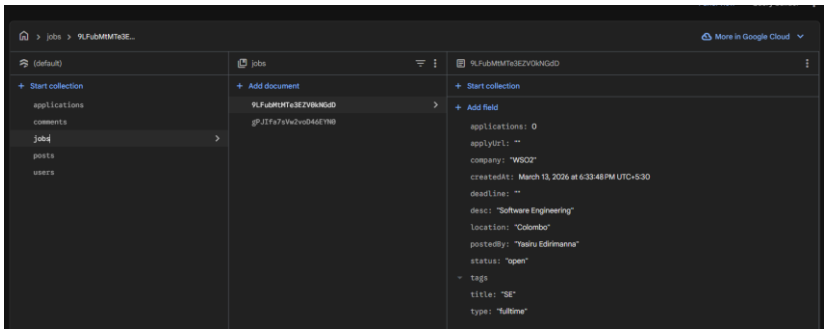


- Jobs - Post and apply for opportunities, saved to Firebase Firestore

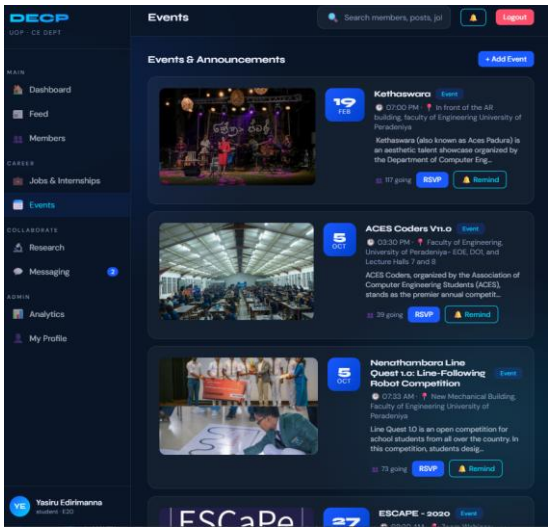




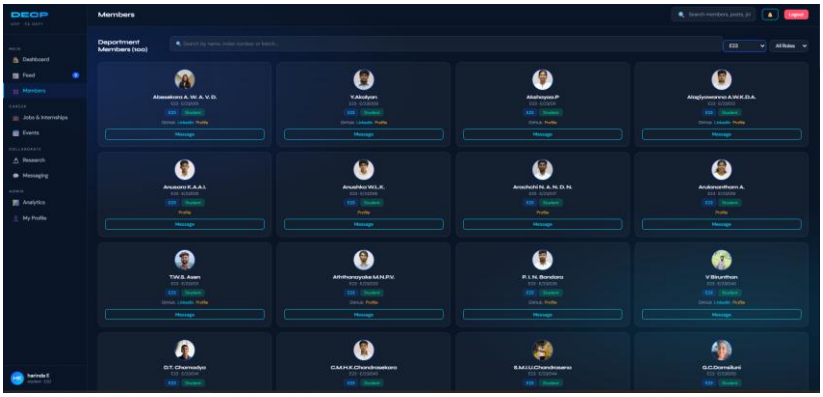
Firestore database

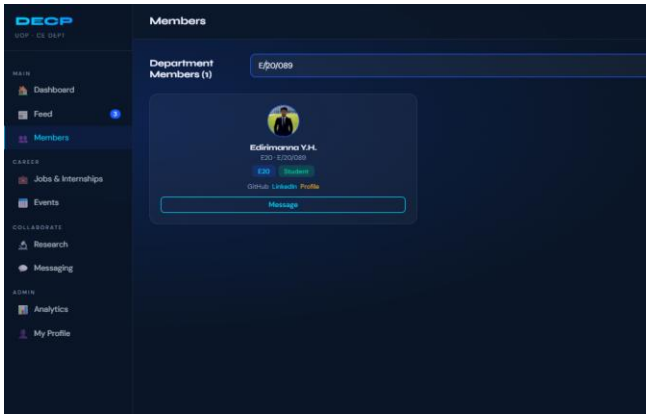


- Events - Real CE department events from portal.ce.pdn.ac.lk API with RSVP

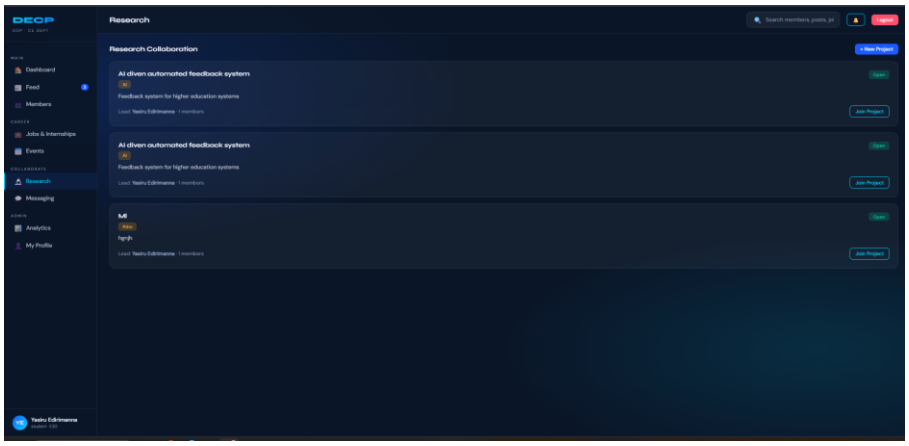


- Members - Real student profiles from api.ce.pdn.ac.lk with photos

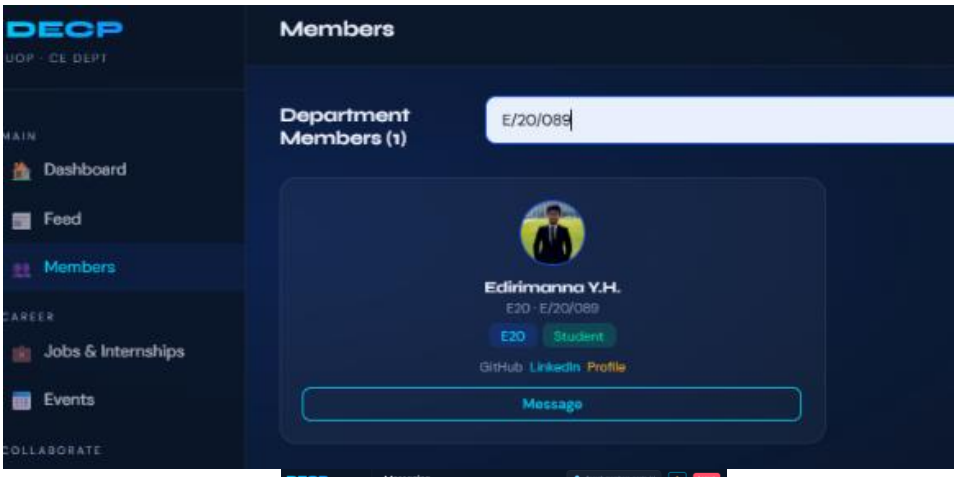


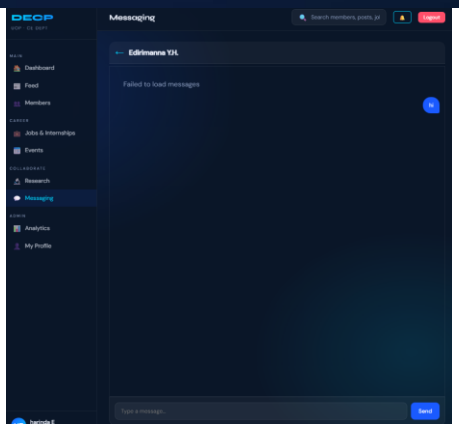


- Research - Create and join research collaboration projects



- Messaging - Direct messaging interface between users

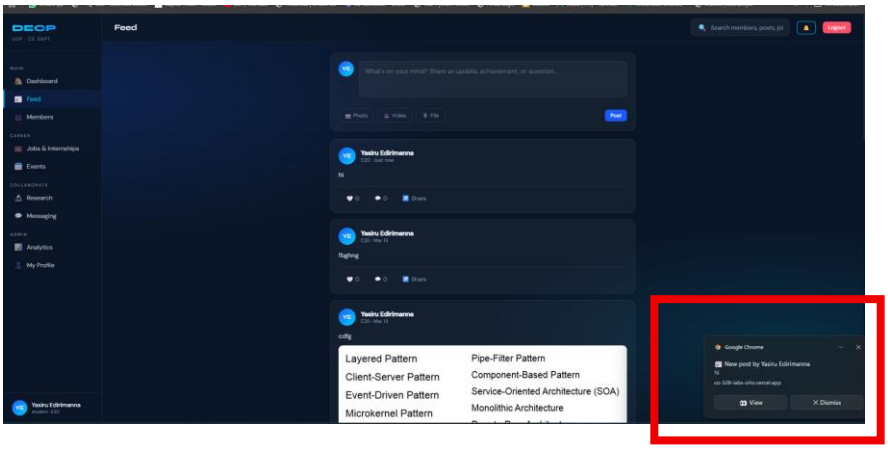




This part is not fully completed just example is their. It need small chages to do

- Analytics - Platform usage metrics and visualizations

- Push Notifications - Browser push notification subscription and delivery



3.3.3 CE Department API Integration

API Endpoint	Data Used
api.ce.pdn.ac.lk/people/v1/students/	Real student batch lists and individual profiles
portal.ce.pdn.ac.lk/api/events/v2/	Live CE department events with images and location
portal.ce.pdn.ac.lk/api/news/v1/	Latest CE department news on dashboard

3.4 Mobile Client Implementation

The mobile client is built using React Native with Expo, implementing a WebView-based approach to deliver the full web platform experience on mobile devices. This architectural decision ensures both clients share identical functionality while avoiding code duplication.

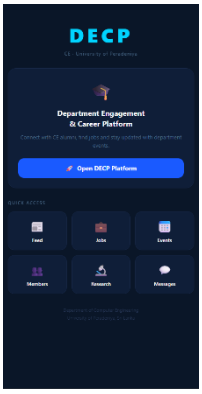
3.4.1 Mobile Architecture Decisions

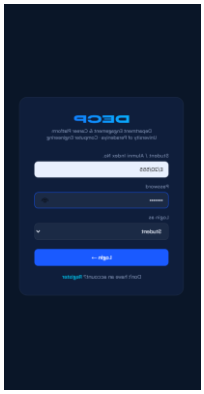
- WebView approach chosen to maximize feature parity with web client
- React Native used for native shell: loading screen, error handling, back button
- Expo framework enables OTA updates without app store re-submission
- Same backend API consumed - no mobile-specific backend changes required

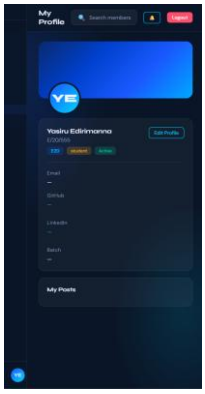
- Android hardware back button handled via React Native BackHandler API

3.4.2 Mobile-Specific Features

- Custom DECP branded loading screen while web content loads
- Network error detection with retry functionality
- Android back navigation support
- Dark status bar matching app theme





4. Research Findings

4.1 Analysis of Similar Platforms

The team researched several real-world platforms to inform DECP's architectural decisions:

4.1.1 LinkedIn

- Uses microservices architecture with hundreds of independent services
- LinkedIn uses Kafka for event-driven communication between services
- Graph-based data model for connection relationships
- Real-time notifications via WebSockets and push notifications
- Relevance to DECP: Job posting workflow, alumni-student connection model, professional feed

4.1.2 Facebook

- GraphQL API allows clients to request exactly the data they need
- News feed uses ranking algorithm - EdgeRank determines what users see
- Media stored on distributed CDN for low-latency delivery globally
- React Native used for mobile app -same framework as DECP mobile
- Relevance to DECP: Feed design, media upload, notification system

4.1.3 University Platforms (Moodle, PeopleSoft)

- Role-based access control critical for academic environments
- Integration with existing student information systems required
- Privacy considerations especially for student data
- Relevance to DECP: Role design, CE API integration approach

4.2 Missing Features and Proposed Improvements

Missing Feature	Proposed Improvement	Priority
Search functionality	Elasticsearch for full-text search across posts, jobs, members	High
Email notifications	SendGrid integration for email digests	Medium
Feed ranking algorithm	ML-based relevance scoring like LinkedIn	Medium
Video streaming	HLS streaming instead of direct video upload	Low
Admin panel	Separate admin dashboard for dept staff	Medium
Offline support	Service Worker caching for offline access	Low

5. Quality Attribute Justifications

5.1 Security

- JWT tokens with 7-day expiry prevent indefinite session access
- Passwords never stored in Firestore handled entirely by Firebase Auth
- Firebase service account key stored as environment variable, never in code
- CORS configured on Express backend to restrict cross-origin requests
- Input validation on all API endpoints prevents injection attacks
- HTTPS enforced on all cloud deployments (Vercel, Render, Firebase)
- Push notification VAPID keys kept server-side never exposed to client

5.2 Scalability

- Firebase Firestore scales horizontally automatically no manual sharding
- Render.com supports scaling to multiple instances under load
- Vercel edge network distributes frontend globally via CDN
- Firebase Storage CDN serves media files from nearest edge location
- Stateless JWT authentication allows any backend instance to verify tokens
- Modular route structure allows individual services to be extracted to microservices

5.3 Maintainability

- Separate route files per domain (auth, posts, jobs, events) easy to locate and modify
- Shared Firebase initialization in single firebase.js file
- Environment variables for all secrets easy to rotate without code changes
- Consistent error handling patterns across all routes
- GitHub version control with descriptive commit messages
- Auto-deployment on push eliminates manual deployment steps

5.4 Availability

- Render free tier has automatic restart on crash
- Firebase has 99.95% SLA highly available database
- Vercel has 99.99% uptime SLA for frontend
- Graceful degradation web client falls back to seed data if API unavailable
- CE Department API failures handled silently with fallback data

5.5 Performance

- Parallel API calls using Promise.all() in dashboard reduces load time
- Pagination on posts feed (limit 20 per request)
- Images served from Firebase Storage CDN not from backend server
- Lazy loading of member profiles batch-by-batch to avoid rate limits
- Client-side caching of user session in localStorage

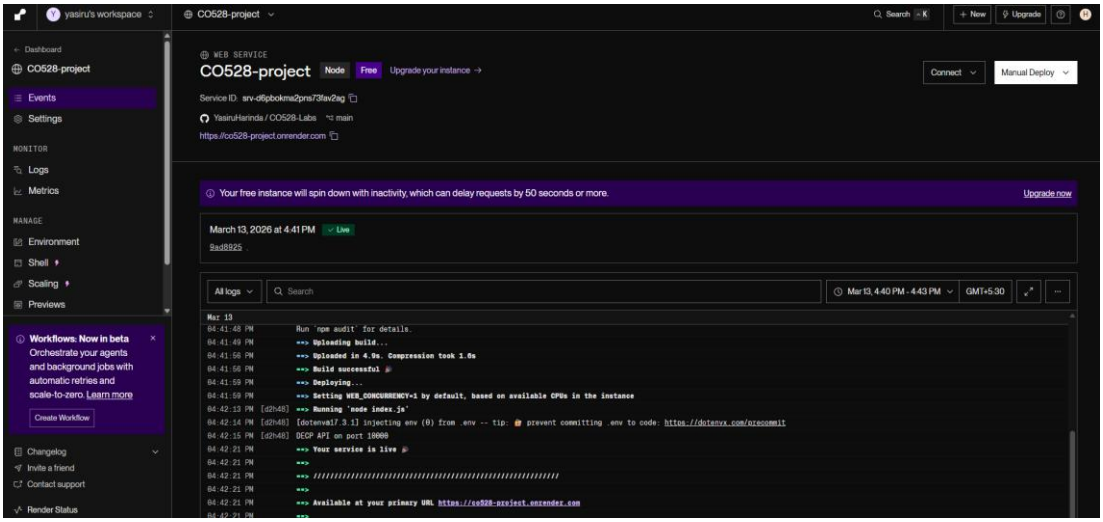
6. Cloud Deployment Details

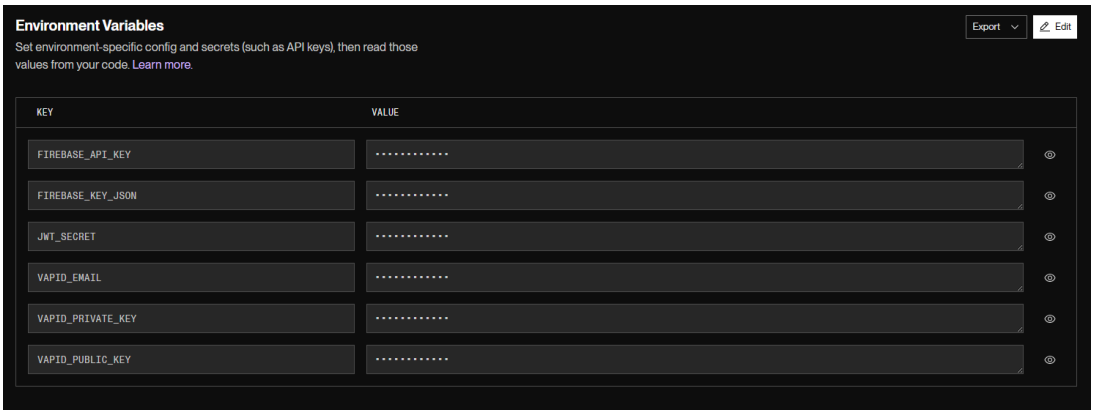
6.1 Backend Deployment (Render.com)

- Platform: Render.com free tier web service
- Runtime: Node.js 22
- Build command: npm install
- Start command: node index.js
- Auto-deploy: enabled - triggers on every GitHub push to main branch
- Root directory set to mini_project/decp-backend/ within monorepo

Environment variables configured on Render:

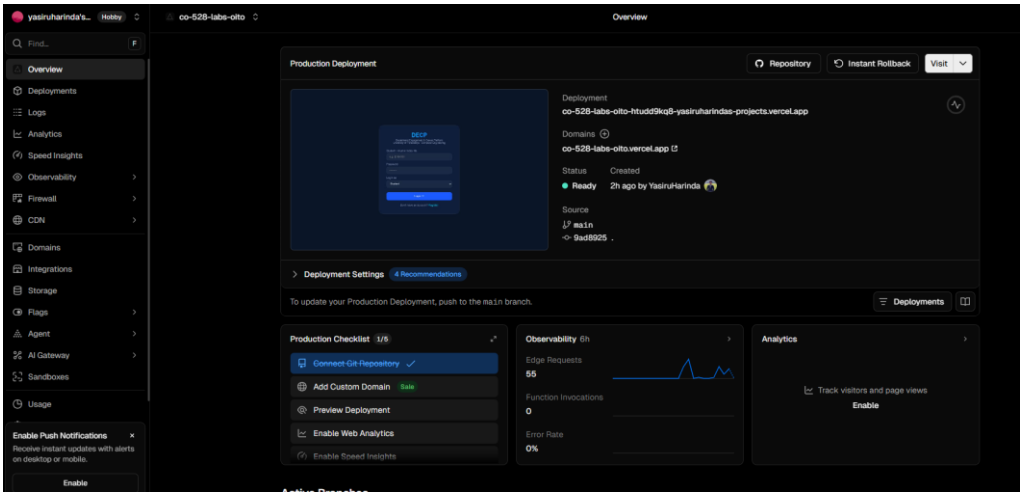
- JWT_SECRET -secret key for JWT token signing
- FIREBASE_API_KEY - Firebase web API key for auth REST calls
- FIREBASE_KEY_JSON - Firebase service account key (full JSON)
- VAPID_PUBLIC_KEY -- Web Push public key
- VAPID_PRIVATE_KEY - Web Push private key
- VAPID_EMAIL - Contact email for push notification service





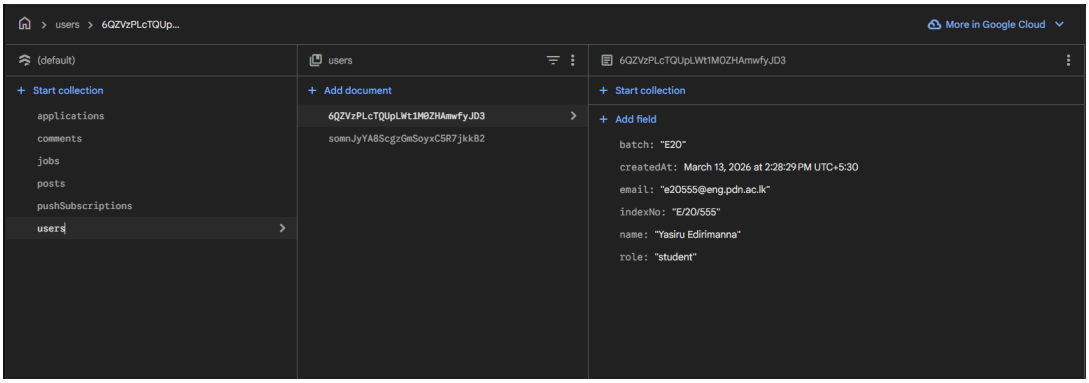
6.2 Frontend Deployment (Vercel)

- Platform: Vercel
- Root directory: mini_project/
- Framework preset: Other (static HTML)
- Build command: empty (no build step needed)
- Auto-deploy: enabled - triggers on every GitHub push
- Global CDN distribution - fast loading worldwide



6.3 Database (Firebase Firestore)

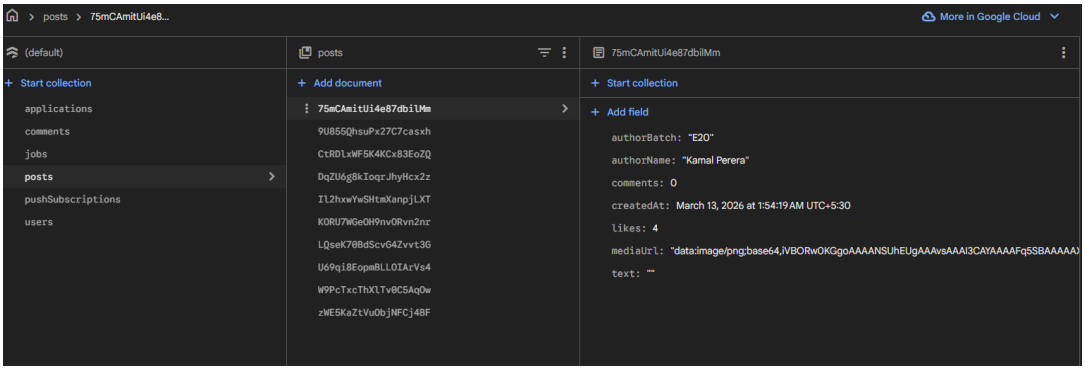
- Region: asia-south1 (Mumbai) - closest to Sri Lanka
- Mode: Started in test mode, Firestore rules updated for production
- Collections: users, posts, comments, jobs, applications, events, research, messages, pushSubscriptions
- Indexes: createdAt desc on posts and jobs collections for ordered queries



6.4 Media Storage (Firebase Storage)

- Region: asia-south1
- Free tier: 5GB storage, 1GB/day download
- Structure: posts/{timestamp}_{filename}
- Access: files made public after upload for direct CDN access

- Supported types: images (jpg, png, gif, webp) and videos (mp4, webm)



6.5 Scalability Considerations

- Firestore scales automatically - no configuration needed for increased load
- Render can be upgraded to paid plan for always-on (no cold start) and multiple instances
- Firebase Storage and Auth scale transparently with usage
- Backend services designed as independent modules - can be split to microservices
- Vercel CDN handles traffic spikes automatically

7. Design Decisions & Justifications

7.1 Why Single HTML File for Web Client

The web frontend is implemented as a single self-contained HTML file rather than using a framework like React or Vue. This decision was made for the following reasons:

- No build step required - deploy by pushing a single file to GitHub.
- Works by simply opening the file - useful for offline demos.
- Faster iteration - edit and refresh without compilation.
- Vercel deployment is trivial with no framework configuration.
- Architecture quality is demonstrated through design, not framework choice.

7.2 Why Firebase over Traditional SQL Database

- Serverless - no database server to manage or provision
- Free tier is sufficient for academic project scale
- Built-in authentication reduces implementation complexity
- Real-time listeners available for future chat/notification features
- Firebase Storage integrated with same SDK - single platform for data + files

7.3 Why WebView for Mobile App

- Guarantees 100% feature parity with web client
- Single codebase for all features - no duplication of business logic
- Fixes and new features automatically available on mobile
- Faster development - mobile team focuses on native shell and UX
- Appropriate for academic demonstration within project timeline

7.4 Why Render over AWS/GCP/Azure

- Free tier requires no credit card - accessible to all team members
- GitHub integration with auto-deploy simplifies CI/CD pipeline
- Sufficient performance for academic project demonstration
- Architecture principles demonstrated are identical to production cloud platforms

7.5 Direct CE API Integration

Rather than copying department data into our own database, DECP fetches directly from the CE Department's public APIs for members, events, and news. This decision:

- Ensures data is always up to date without synchronization complexity
- Respects the department's data as the single source of truth
- Reduces storage requirements and GDPR/privacy concerns
- Demonstrates real integration architecture used in enterprise systems

8. Setup & Deployment Guide

8.1 Prerequisites

- Node.js 18 or above
- npm 9 or above
- Firebase account (free)
- GitHub account
- Render.com account (free)
- Vercel account (free)

8.2 Local Development Setup

Backend

```
cd mini_project/decp-backend

npm install

# Create .env file with required variables

npm run dev # Backend runs at http://localhost:3000
```

Frontend

```
# Open mini_project/index.html in browser

# Or use Live Server extension in VS Code
```

Mobile

```
cd mini_project/DECP-Mobile

npm install

npx expo start --clear

# Scan QR code with Expo Go app on phone
```

8.3 Required Environment Variables

Variable	Description
JWT_SECRET	Secret string for signing JWT tokens
FIREBASE_API_KEY	Firebase web API key (from Firebase Console > Project Settings)
FIREBASE_KEY_JSON	Full Firebase service account JSON (from Service Accounts tab)
VAPID_PUBLIC_KEY	VAPID public key for push notifications
VAPID_PRIVATE_KEY	VAPID private key for push notifications
VAPID_EMAIL	Contact email (format: mailto:your@email.com)

8.4 Links

- Live Web App: https://co-528-labs-oito.vercel.app/
- Backend API: https://co528-project.onrender.com
- GitHub Repository: https://github.com/YasiruHarinda/CO528-Labs
- Firebase Console: https://console.firebase.google.com

9. Conclusion

The Department Engagement & Career Platform (DECP) successfully demonstrates the application of Software Architecture principles in a real-world inspired system. The project implements a Service-Oriented Architecture with eight distinct services, each with well-defined REST API boundaries.

Key architectural achievements include:

- Complete SOA implementation with 8 independent route-based services
- Web-Oriented Architecture with a Single Page Application consuming backend APIs
- Mobile Architecture sharing the same backend via React Native + Expo
- Cloud Architecture with live deployment on Render (backend), Vercel (frontend), and Firebase (database + storage)
- Enterprise Architecture integrating with real CE Department APIs for live data
- Product Modularity with clearly separated core and optional feature modules

The platform integrates real data from the CE Department's public APIs, providing authentic student profiles, live events, and department news. Authentication is handled through Firebase Auth with JWT tokens, and media files have persisted in Firebase Storage with public CDN URLs.

Future improvements would include real-time WebSocket messaging, an ML-based feed ranking algorithm, a dedicated admin panel for department staff, and full microservice extraction for each service domain.